



Universidad
Indoamérica

UNIT 2 · FUNDAMENTALS OF OPERATING SYSTEMS

System calls and low-level I/O

Universidad Tecnológica Indoamérica Lab TD1

Abstraction

- Simplify and standardise
- Portability of the kernel and of programs (POSIX)
- Make driver development easier
- Stabilise the execution environment

The abstractions of the course

Files (U3) · processes and virtual memory (U4) · communication (U5, U6, U8) · threads (U7) · and now agents (U9)

Protection

- Identification and usage/access rights
- Isolation between processes and between users
- Data encryption
- Security policies (network, files...)

2026: the same principles apply to a new "user": the **AI agent** (its own identity, least privilege, auditing — U9).

Virtualisation

Portability and programming

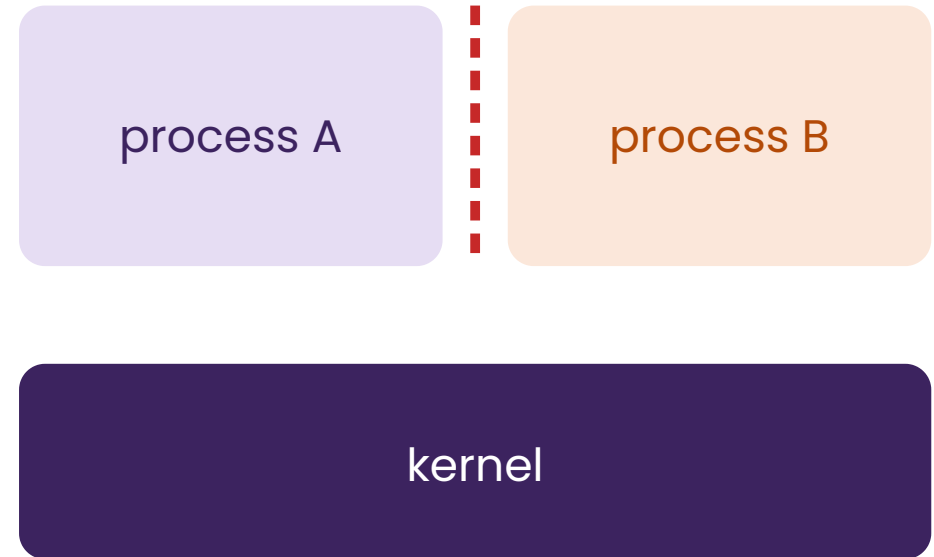
- VFS: a single interface for ext4, Btrfs, NFS, /proc ...
- A homogeneous view of the hardware (USB, SATA and NVMe disks seen the same way)
- POSIX standardises the system functions

Stability and evolution

- Process isolation: one failure does not bring the machine down
- Simulating a machine or another OS: QEMU/KVM, WSL 2
- Containers: namespaces + cgroups
- IPv4 over IPv6

The process concept

- Every process has its own **context**
- A **private** view of memory: it cannot see the others'
- It is isolated and **believes it is alone**: the other processes and the system itself are hidden



Two execution modes of the processor

User mode (non-privileged)

The default. The program only adds, reads and writes **within its own memory**.

Kernel mode (privileged)

For anything else the process **asks for permission** with a special instruction—a system call—: open, modify or rename a file, allocate memory, create or end a task...

The hardware switches mode and the OS checks that the operation is allowed: anything that could harm other tasks or users.

Kernel mode and the kernel

- Some instructions (memory management, interrupts) are **forbidden** in user mode
- The kernel **manages** processes, but **is not a process**
- It manages resources and hardware access; it always resides in memory; it is loaded at boot (U1, A1.3)
- A user process **switches to kernel mode** to use a kernel service

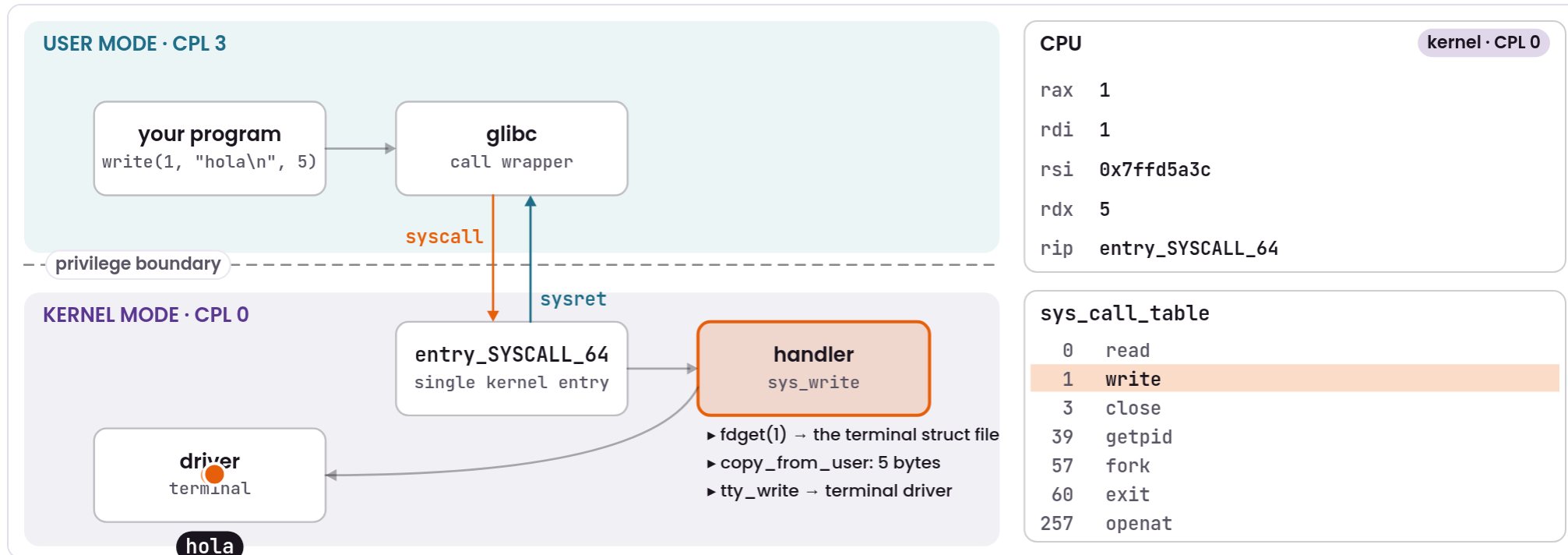
System calls: the interface

- Interface between user mode and kernel mode
- The mode switch is triggered by an instruction: `ta` (Sparc), `int 0x80` (old x86), today `syscall` (x86-64), `svc #0` (ARM64), `ecall` (RISC-V)
- The kernel has an entry handler and a call table

System calls are costly!

The original said "several hundred cycles". Measured on the course machine: $\approx 0.7 \mu\text{s}$ per `read` + `write` iteration (thousands of cycles).

The journey of `write(1, "ho!a\n", 5)`



The instruction, seen with `objdump`

REGISTER	WRITE(1, BUF, 5)
<code>rax</code>	1 (number of <code>write</code>)
<code>rdi</code>	1 (descriptor)
<code>rsi</code>	address of <code>buf</code>
<code>rdx</code>	5

Real output

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-
gnu/libc.so.6 | grep -A6 -E '^[0-9a-f]+'
<__write_nocancel@@'
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:
129060:    endbr64
129064:    mov     $0x1,%eax
129069:    syscall
12906b:    cmp     $0xffffffffffffffff,000,%rax
129071:    ja      129078
<__write_nocancel@@GLIBC_PRIVATE+0x18>
129073:    ret
```

Errors: -1, errno and perror

- A failing call returns **-1** and leaves the code in **errno**
- Message: `void perror(const char *s);` or **%m** in `fprintf` (glibc extension)
- In the course: `exit_if(rc == -1, "write")`

Real output

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

C

```
ssize_t rc = write(STDOUT_FILENO, buf, n);
if (rc == -1)
    fprintf(stderr, "write: %m\n");
exit_if(rc == -1, "write");
```

Tracing the calls: **strace**

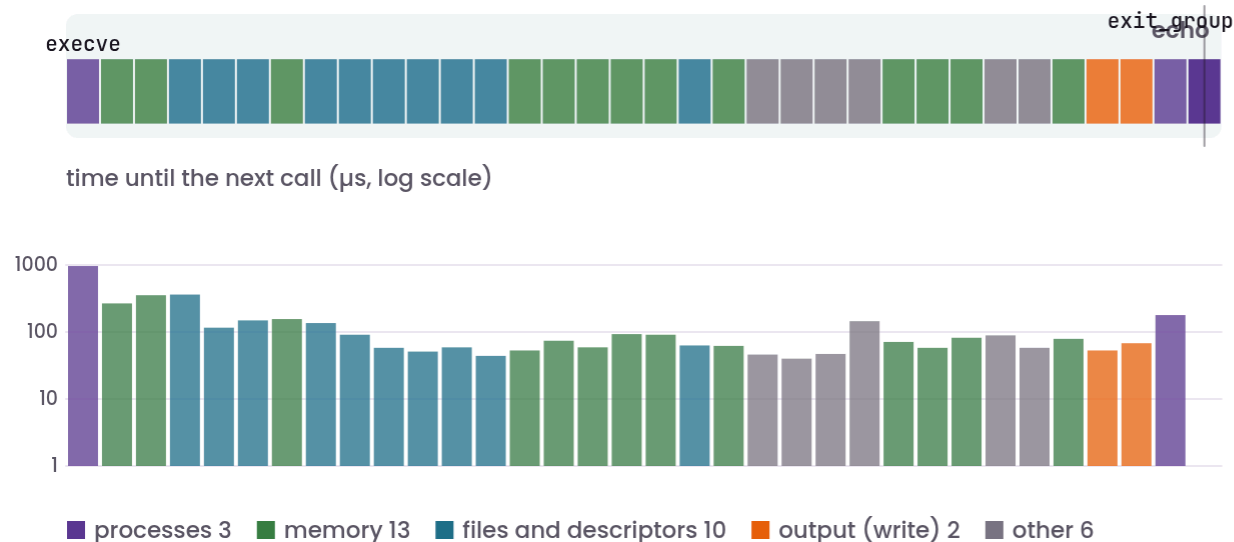
Real output

```
$ strace -r ./echo hola > /dev/null      # the whole life of the process
0.000000 execve("./echo", [".echo", "hola"], 0x7ffff5a76120 /* 129 vars */) = 0
0.000965 brk(NULL)                      = 0x615cdaf80000
0.000267 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x747beb566000
0.000354 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
0.000362 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
0.000116 fstat(3, {st_mode=S_IFREG|0644, st_size=108847, ...}) = 0
0.000149 mmap(NULL, 108847, PROT_READ, MAP_PRIVATE, 3, 0) = 0x747beb54b000
0.000156 close(3)                       = 0
...
```

Real output

```
0.000089 getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_NONBLOCK) = 8
0.000058 munmap(0x747beb54b000, 108847) = 0
0.000079 write(1, "hola", 4)              = 4
0.000053 write(1, "\n", 1)                = 1
0.000068 exit_group(0)                  = ?
0.000179 +++ exited with 0 +++
```

The whole trace, call by call



```

set_tid_address(0x747beb548d68) = 809641
set_robust_list(0x747beb548a20, 24) = 0
rseq(0x747beb548680, 0x21, 0, 0x53053053) = 0
mprotect(0x747beb40e000, 16384, PROT_READ) = 0
mprotect(0x615caa96a000, 4096, PROT_READ) = 0
mprotect(0x747beb5ab000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, ...
getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_...
munmap(0x747beb54b000, 108847) = 0
write(1, "hola", 4)      = 4
write(1, "\n", 1)       = 1
exit_group(0)           = ?
+++ exited with 0 +++

```

System calls versus library functions

System calls (man 2)

Inodes: `stat`, `unlink`, `chmod`, `link` ...

Descriptors: `open`, `read`, `write`, `lseek`,
`close`, `fcntl` ...

C library (man 3)

`fopen`, `fread`, `fprintf`, `fclose` ... **are not system calls**: they use `FILE *` (with a buffer) instead of a descriptor. `fileno()` converts one into the other.

Reading data: `read`

C

```
ssize_t read(int fd, void *buf, size_t count);
```

- Reads **up to** `count` bytes from the file of `fd` into `buf`, which must already be allocated (`count` $\leq$ its size)
- Returns the bytes read, **0 at end of file** or -1 on error
- **It may return fewer** bytes than requested: pipe, terminal, network or a signal (`EINTR` if it read nothing)

Writing data: `write`

C

```
ssize_t write(int fd, const void *buffer, size_t count);
```

- Writes the first `count` bytes of `buffer` to the file of `fd`
- Returns `-1` on error or the bytes written

Fix to the original: "if it writes less than `count`, it is an error" is not always true. With full pipes or sockets, or a signal half-way, the write can be **partial and valid**: repeat with the rest (`write_all`).

Where is the bug?

C

```
/* slide 51 of the original */
while ((byte = read(fd, buf, length))  $\neq$  0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;    /* BUG */
}
```

C

```
/* fixed: advance by what was READ */
while (length > 0) {
    byte = read(fd, buf, length);
    if (byte == -1 && errno == EINTR)
        continue;
    exit_if(byte == -1, "read");
    if (byte == 0)    /* end of file */
        break;
    length -= byte;
    buf += byte;
}
```

The same loop with a file and with a pipe

Original (slide 51)

```
while ((byte = read(fd, buf, length)) != 0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;
}
```

buf = 43 length = 0

\$ strace -e trace=read ./prog (pipe)

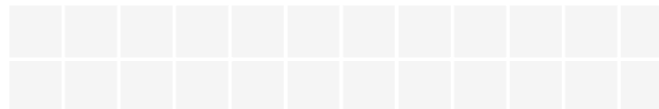
read(3, buf+36, 16) = 9

read(3, buf+43, 7) = 7

read(3, buf+43, 0) = 0

pipe: the writer does 4 write() of 4 + 4 + 9 + 7 bytes

pipe



process memory: char buf[24] and what comes after it

+0 r e a d

+12 () m

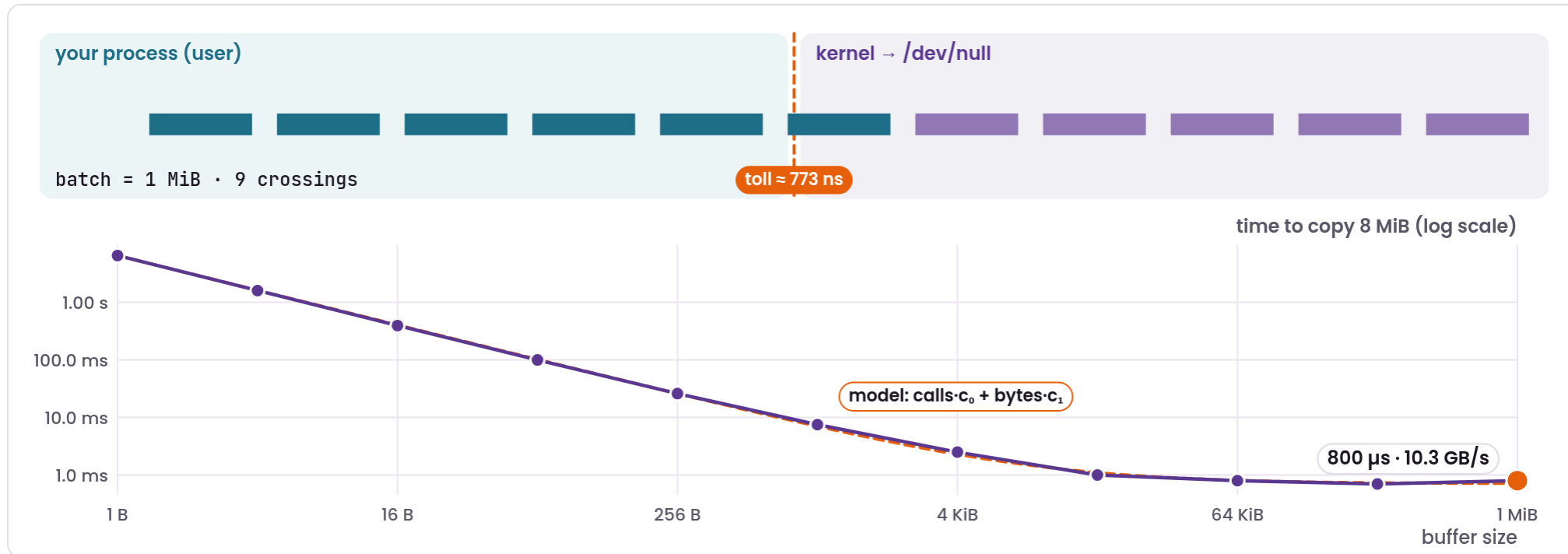
end of buf[24] · neighbouring memory: other variables, return address...

+24 ? ? ? ? ? ? ? ? ? ? ? ?

+36 a y r e t u l e s s

buf = 43

How much does every crossing cost? Measured



Fewer crossings, same work

vDSO

`clock_gettime` without entering the kernel.

io_uring

Shared queues:
hundreds of operations per call.

eBPF

Observe calls system-wide without stopping processes.

Solved TD1 + AI challenge

- Build and run the 7 exercises; break one on purpose
- Explain every line of `strace ./echo`
- AI challenge: measure the cost of calls on your machine

Bash

```
make -C tds && tds/tests/run.sh td1
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. Why is the kernel not a process, and why is it not root?
2. What does `read` return at end of file and what does glibc do with -9?
3. What was wrong with the example on slide 51?